

Gestione Sartoria per Suite Eleganti

Descrizione dei metodi per ciascun modulo Python

capo_principale.py

Classe CapoPrincipale(ABC)

È la classe astratta base per tutti i capi principali. Eredita da **ABC** (importato da `abc`). Nel `__init__` riceve e memorizza come attributi privati: **codice**, **nome**, **tessuto**, **colore**, **taglia** e **prezzo**. Tutti gli attributi devono essere accessibili tramite `@property`. Per **prezzo** e **taglia** aggiungi anche il setter, perché sono i campi che possono essere modificati.

descrizione() [abstractmethod]

Metodo astratto decorato con `@abstractmethod`. Non ha corpo. Ogni sottoclasse è obbligata a implementarlo restituendo una stringa che descrive il capo includendo tutte le sue caratteristiche specifiche.

calcola_prezzo() [abstractmethod]

Anch'esso astratto. Ogni sottoclasse lo implementa calcolando il prezzo finale a partire da `self.prezzo` più eventuali maggiorazioni legate alla personalizzazione (ad esempio il numero di bottoni per la giacca).

__str__()

Restituisce una stringa leggibile con le informazioni comuni a tutti i capi: codice, nome, tessuto, colore, taglia e prezzo. Può chiamare `self.calcola_prezzo()` internamente così il prezzo mostrato è già quello finale comprensivo di maggiorazioni.

Classe Giacca(CapoPrincipale)

Ha un attributo aggiuntivo proprio: **numero_botoni** (intero), che riceve nel `__init__` insieme a tutti gli attributi della classe base, che passa a `super().__init__()`.

descrizione()

Restituisce una stringa con tutte le informazioni base più il numero di bottoni. È la versione concreta del metodo astratto della classe padre.

calcola_prezzo()

Restituisce il prezzo base più una maggiorazione calcolata in base al numero di bottoni (ad esempio un costo fisso per ogni bottone oltre una soglia base).

Classe Pantalone(CapoPrincipale)

Ha un attributo aggiuntivo: **tipo_taglio** (stringa, ad esempio "slim", "regular", "wide").

descrizione()

Restituisce le informazioni base più il tipo di taglio.

calcola_prezzo()

Restituisce il prezzo base eventualmente maggiorato in base al tipo di taglio (ad esempio "slim" comporta un sovrapprezzo).

Classe Gilet(CapoPrincipale)

Ha un attributo aggiuntivo: **rever_presente** (booleano, indica se il gilet ha il rever).

descrizione()

Restituisce le informazioni base più l'indicazione sulla presenza del rever.

calcola_prezzo()

Restituisce il prezzo base più una maggiorazione fissa se **rever_presente** è True.

componente_finitura.py

Classe ComponenteFinitura(ABC)

Classe astratta base per tutti i componenti di finitura. Nel `__init__` riceve: **codice**, **nome**, **materiale**, **colore** e **prezzo**. Tutti accessibili tramite `@property`, con setter per **prezzo**.

descrizione() [abstractmethod]

È l'unico metodo astratto di questa gerarchia. Ogni sottoclasse lo implementa restituendo una stringa con le informazioni del componente inclusa la sua caratteristica propria.

__str__()

Restituisce una stringa con codice, nome, materiale, colore e prezzo.

Classe Cravatta(ComponenteFinitura)

Attributo proprio: **larghezza** (float, in centimetri).

descrizione()

Restituisce le informazioni base più la larghezza in centimetri.

Classe Papillon(ComponenteFinitura)

Attributo proprio: **tipo_chiusura** (stringa, ad esempio "elastico", "regolabile", "fissa").

descrizione()

Restituisce le informazioni base più il tipo di chiusura.

Classe Pochette(ComponenteFinitura)

Attributo proprio: **piega_decorativa** (stringa, ad esempio "piatta", "a punta", "a ventaglio").

descrizione()

Restituisce le informazioni base più il tipo di piega decorativa.

sartoria.py

Classe Sartoria

Il gestore centrale del sistema. Nel `__init__` inizializza due liste vuote: `self.capi` (per oggetti di tipo **CapoPrincipale**) e `self.componenti` (per oggetti di tipo **ComponenteFinitura**).

aggiungi_capo(capo)

Riceve un oggetto già costruito di tipo **CapoPrincipale** e lo appende a `self.capi`.

rimuovi_capo(codice)

Scorre `self.capi` cercando un capo con quel codice, lo rimuove e restituisce `True`. Se non lo trova, stampa un messaggio e restituisce `False`. Ricorda: non usare `else` dentro il `for`; gestisci il caso "non trovato" dopo il ciclo.

modifica_capo(codice, ...)

Riceve il codice del capo da modificare e i nuovi valori (almeno **prezzo** e **taglia**). Usa `cerca_capo()` per trovare l'oggetto e aggiorna i suoi attributi tramite i setter. Restituisce `True` se lo trova, `False` altrimenti.

cerca_capo(codice)

Scorre `self.capi` e restituisce l'oggetto il cui codice corrisponde. Se non lo trova restituisce `None`. Viene usata internamente da `modifica_capo()` e `rimuovi_capo()` per evitare di duplicare il ciclo di ricerca.

aggiungi_componente(comp)

Riceve un oggetto di tipo `ComponenteFinitura` e lo appende a `self.componenti`.

analizza_tutti()

Scorre `self.capi` e per ogni elemento chiama `capo.descrizione()` e `capo.calcola_prezzo()`. Poiché la lista contiene oggetti misti (Giacca, Pantalone, Gilet), Python chiama automaticamente la versione corretta di ciascun metodo grazie al polimorfismo.

analizza_per_tipo(tipo)

Riceve una stringa che rappresenta il tipo (ad esempio "Giacca"). Scorre `self.capi` e usa `isinstance(capo, Giacca)` per filtrare solo quelli del tipo richiesto, poi chiama `descrizione()` su ciascuno.

analizza_per_tipo_e_pers(tipo, valore)

Riceve il tipo e un valore specifico della personalizzazione (ad esempio "Giacca" e "6" per le giacche con 6 bottoni). Filtra prima per tipo con `isinstance()`, poi verifica che l'attributo proprio corrisponda al valore cercato, e stampa solo i capi che soddisfano entrambe le condizioni.

main.py

Non contiene classi. All'avvio crea un oggetto `Sartoria` e un loop principale con il menu.

crea_capo(sartoria)

Chiede all'utente che tipo di capo vuole creare (Giacca, Pantalone o Gilet) con un sottomenu, raccoglie tutti i dati necessari tramite `input()`, costruisce l'oggetto corrispondente e lo passa a `sartoria.aggiungi_capo()`.

modifica_capo(sartoria)

Chiede il codice del capo da modificare e i nuovi valori di prezzo e taglia, poi chiama `sartoria.modifica_capo()`. Se il capo non esiste, stampa un messaggio di errore.

elimina_capo(sartoria)

Chiede il codice del capo da eliminare e chiama `sartoria.rimuovi_capo()`.

genera_dati(sartoria)

Usa il modulo `random` per creare automaticamente un insieme di capi e componenti con valori casuali e li aggiunge alla sartoria. Serve a popolare il sistema rapidamente per testare le analisi senza inserire tutto manualmente.

menu_analisi(sartoria)

Mostra un sottomenu con le tre opzioni di analisi e chiama il metodo corrispondente di `sartoria`: `analizza_tutti()`, `analizza_per_tipo()` (chiedendo prima all'utente quale tipo), oppure `analizza_per_tipo_e_pers()` (chiedendo tipo e valore della personalizzazione).

main()

Inizializza `sartoria = Sartoria()`, mostra il menu principale in un loop e smista verso le funzioni corrette in base alla scelta dell'utente. Alla fine del file va inserito `if __name__ == "__main__": main()`.